Simple Chat Application

Security posture, controls, and operational practices

Generated: February 24, 2026

Overview

The **Simple Chat Application** is a web-based platform designed to facilitate secure, context-aware interactions with generative AI models using **Azure OpenAI**. Its primary capability is **Retrieval-Augmented Generation (RAG)**, enabling users to ground conversations in user-provided content. Users can upload documents to personal (*Your Workspace*) or shared (*Group Workspaces*) areas; documents are processed via Azure document extraction, chunked, embedded, and indexed to support hybrid retrieval (semantic + keyword).

The application supports optional features that can be enabled by administrators, such as content safety review, image generation, video/audio processing for RAG, document classification, user feedback collection, conversation archiving, AI-driven metadata extraction, and enhanced citations linked to source documents in cloud storage.

The solution typically stores conversations and metadata in a managed database service and uses centralized identity (Microsoft Entra ID) with role-based access control (RBAC). It is intended for enterprise deployments and supports both Azure Commercial and Azure Government environments.

Security objectives

- **Confidentiality:** Protect user prompts, documents, embeddings, retrieval results, and generated outputs from unauthorized access.
- **Integrity:** Prevent unauthorized modification of configuration, documents, indexes, and conversation records; ensure citations and lineage are trustworthy.
- **Availability:** Maintain resilient service operation under expected load and abuse scenarios (e.g., DoS, ingestion spikes).
- **Governance:** Ensure risky features (e.g., uploads, sharing, archiving, safety review) are controlled via explicit approvals and least privilege.
- **Auditability:** Provide sufficient logging and retention to support investigations, compliance, and incident response.

Scope and assumptions

- **In scope:** web client, backend APIs, identity integration, document ingestion and indexing, storage, admin settings, approvals, CI/CD, and infrastructure configuration.
- **Out of scope:** third-party systems not controlled by the organization (except where integration controls are described), and physical security.
- **Assumptions:** TLS is enforced for all endpoints; secrets are stored in a secrets manager; RBAC is enforced server-side; logs are retained per policy.

Architecture and data flows (high level)

Core RAG workflow:

1. User authenticates via Entra ID and receives short-lived tokens with role claims.
2. User uploads a document to a workspace or an ephemeral session store.
3. The backend validates file type/size, stores content in Azure Storage, extracts content, chunks, embeds, and indexes content for retrieval. Malware scanning (if desired) is provided by Defender for Storage when enabled by the organization on the storage account.
4. For a chat request, the backend retrieves authorized chunks, constructs a grounded prompt, and calls the model.
5. Responses are returned with citations; messages and audit metadata are stored per retention policy.

Architecture diagram

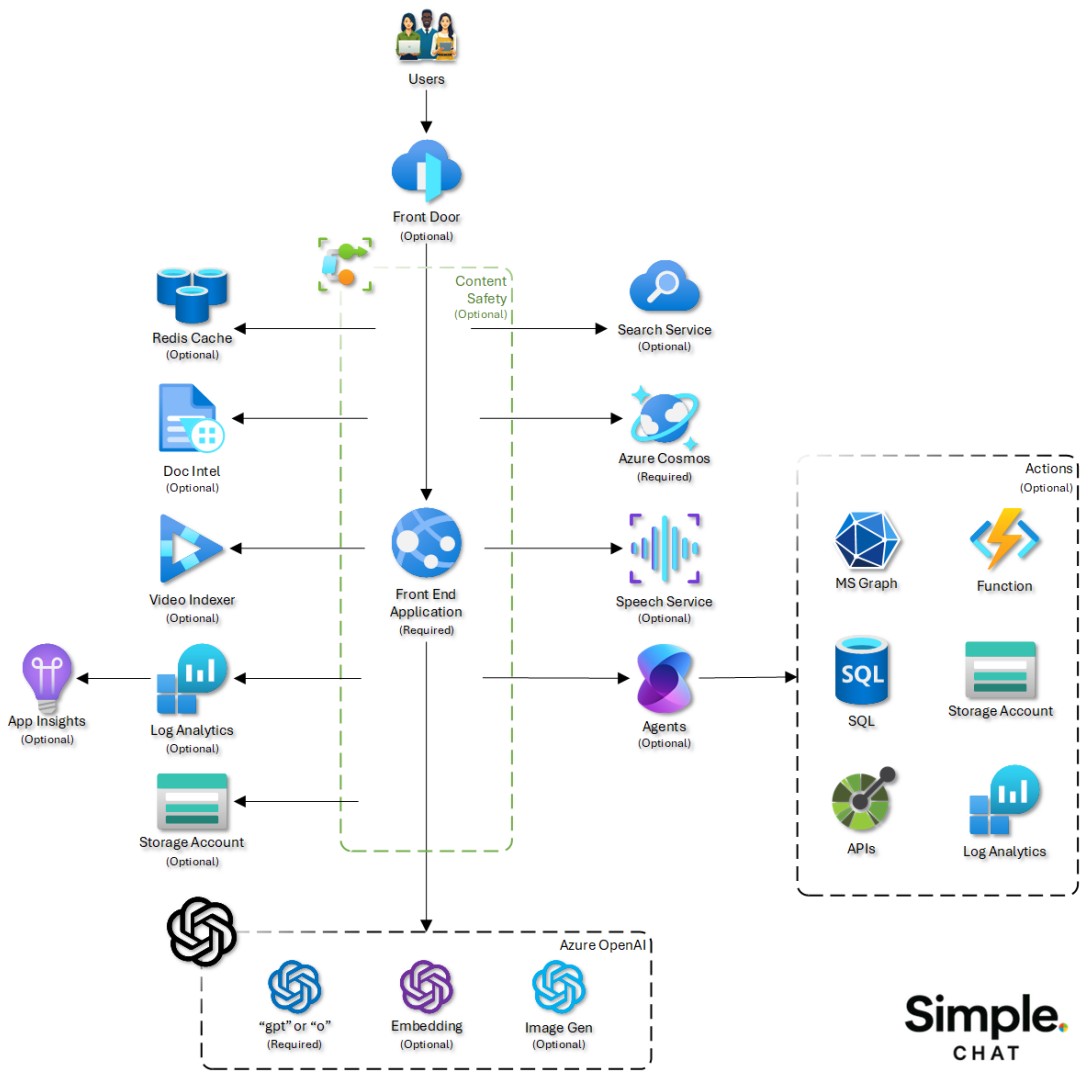

Figure 1: Application architecture overview.

Workflow: upload and workspace RAG

This workflow shows how user documents move through storage, extraction, embeddings, and retrieval, and highlights where authorization and optional malware scanning (Defender for Storage) apply.

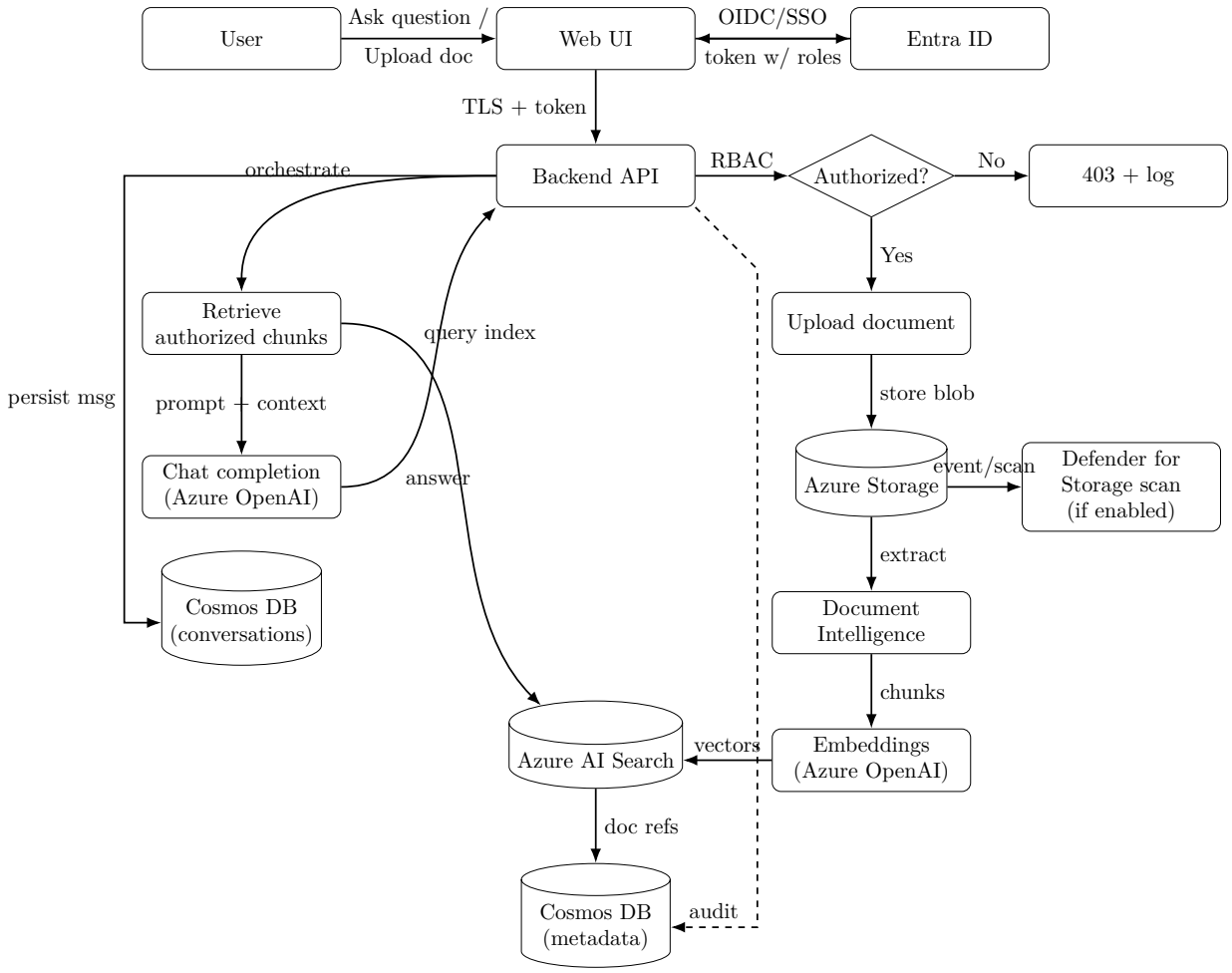

Figure 2: Workflow: upload a document and use workspace RAG, including optional Defender for Storage scanning.

Workflow: web search grounding (optional)

When web search is enabled and a user invokes it, the system retrieves web results via an agent tool and uses them as grounded context alongside the user prompt and (optionally) workspace context.

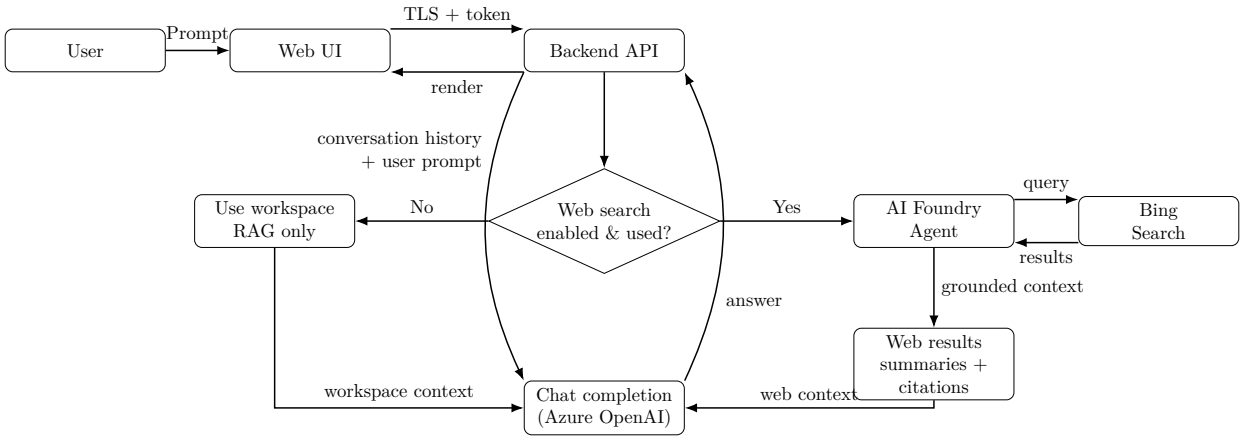

Figure 3: Workflow: web-search grounded responses using an agent tool with Bing (optional feature).

Services in the architecture and security roles

- **Web Application (UI) and Backend API (App Service / hosting):** terminates TLS for user traffic, validates identity tokens, enforces authorization on every request, and orchestrates RAG and downstream service calls.
- **Microsoft Entra ID (Azure AD):** provides authentication (OIDC/OAuth2) and role claims used for RBAC decisions (admin vs. user capabilities, workspace access, and governance features).
- **Azure AI Search:** stores the *search index* containing chunked document content (and associated metadata) for hybrid retrieval; access is restricted to the backend using least-privilege credentials/managed identity, and data is encrypted at rest.
- **Azure Storage Accounts:** store source documents and derived artifacts (e.g., extracted text/chunks/citation blobs) depending on enabled features; access is scoped by container/prefix and controlled via RBAC, private endpoints (where used), and short-lived signed access (SAS) when direct access is required. Malware scanning is provided by **Microsoft Defender for Cloud** (Defender for Storage) when enabled and assigned to the storage account; the application does not perform explicit antivirus scanning.
- **Azure Cosmos DB:** stores conversations and messages, document parent metadata for all workspaces, application configurations, and user configurations; access is limited to the backend with least privilege and encrypted at rest.
- **Azure OpenAI (LLM + Embeddings):** used to generate embeddings for indexing and to produce chat completions; requests are made over TLS, and prompts are constructed to include only authorized retrieval context.
- **Azure AI Document Intelligence (optional but typical for ingestion):** extracts text/layout from uploaded documents for chunking and indexing; access is restricted to the backend and uses TLS in transit.

- **Web search (optional): Azure AI Foundry Agent + Bing:** when enabled and invoked, the backend uses an agent tool to retrieve web results (Bing) and incorporates them as grounded context (separately from workspace RAG) under policy controls.

Encryption in transit and at rest (AES-256 where applicable)

- **In transit:** all user-to-app and app-to-service traffic uses TLS. Where TLS 1.3 is negotiated, AES-GCM cipher suites may be used; where TLS 1.2 is used, modern AES-GCM suites are typical. (Cipher negotiation depends on platform policy and client/server capabilities.)
- **At rest:** Azure Storage and Azure Cosmos DB support encryption at rest using AES-256 by default; Azure AI Search supports encryption at rest and can be configured with customer-managed keys (CMK) for additional assurance.
- **Key management:** keys are managed by Azure-managed keys by default; for higher assurance, CMK can be used with Azure Key Vault to enable key rotation and explicit revocation workflows.

Data locations, retention, and deletion

Where data resides

- **Azure AI Search index:** chunked document content and retrieval metadata used for workspace RAG.
- **Azure Storage Accounts:** original documents and derived artifacts required for citations, reprocessing, or governance workflows (feature dependent).
- **Azure Cosmos DB:** conversations, messages, document parent metadata (across all workspaces), application configs, and user configs.

Retention policy and scheduled deletion

Organizations configure retention policies for documents and conversations. A scheduled job enforces these policies and performs deletions **daily at 2:00 AM** (environment local time as configured). Deletion includes removing eligible content from the relevant stores (Cosmos DB records, storage blobs, and associated search index entries), with audit logs retained according to policy.

Workflow: retention policy enforcement

This workflow documents how the daily retention job identifies eligible items and removes data from Cosmos DB, Storage, and Azure AI Search.

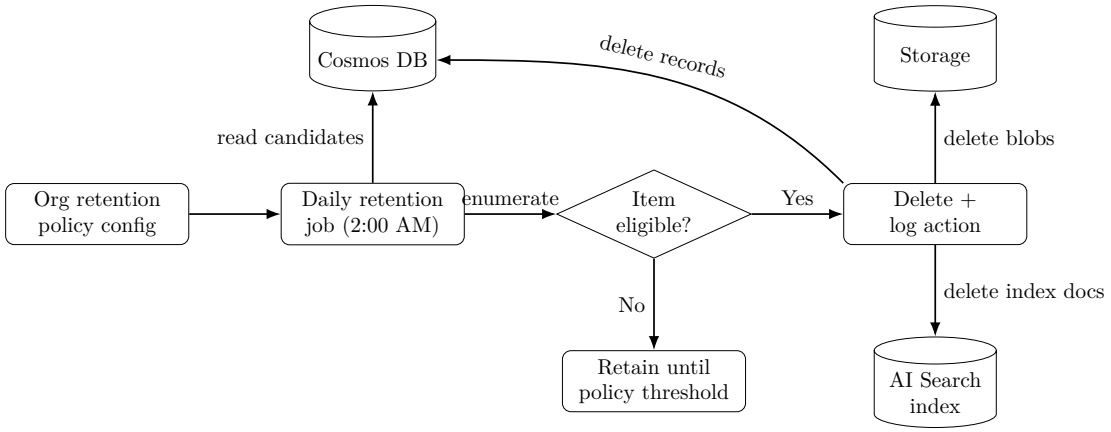

Figure 4: Workflow: retention policy enforcement and scheduled deletion (daily at 2:00 AM).

What is sent to the LLM

For each chat completion request, the application sends:

- the user prompt;
- relevant conversation history (as needed for continuity and policy);
- optional grounded context from **workspace RAG** (retrieved from Azure AI Search) when workspace search is used;
- optional grounded context from **web RAG** when web search is enabled and used (retrieved via the AI Foundry agent tool with Bing as the search provider).

The backend enforces access controls so that only content the user is authorized to access can be retrieved and included as model context.

Administrative governance workflows

Administrative workflows control optional features, retention settings, and policy toggles; these actions are restricted by RBAC and are auditable.

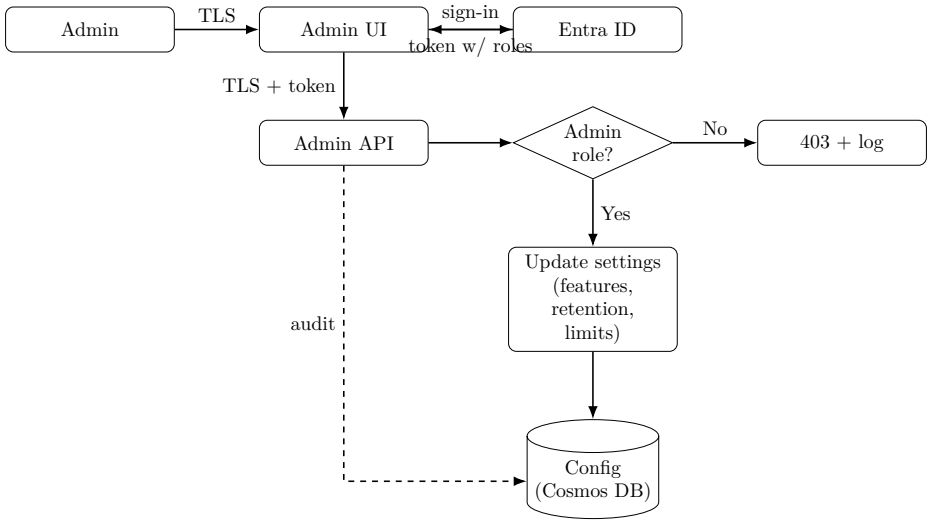

Figure 5: Workflow: admin settings updates with RBAC and auditing.

Threat model overview (high level)

Category	Representative threats	Primary mitigations
Identity & access	Account takeover, token theft, privilege escalation	SSO/MFA, short-lived tokens, RBAC, session protections, audit trails
APIs & services	Broken access control, injection, SSRF, DoS	AuthZ on every endpoint, input validation, rate limiting, allowlists, WAF
Client security	XSS, supply-chain, unsafe script loading	CSP, no external JS, SRI (if needed), dependency scanning, output encoding
File handling	Malware, zip bombs, polyglots, parsing exploits	Allowlist, size caps, AV scanning, sandboxing, isolated storage, quotas
Data protection	Breach, key compromise, leakage in logs	Encryption at rest, KMS/HSM, redaction, least privilege, secure backups
Ops & infra	Misconfiguration, secret leakage, lateral movement	IaC/policy scanning, secret management, network segmentation, monitoring

Identity, authentication, and authorization

Authentication

- Centralized identity via OIDC/SSO (Microsoft Entra ID) with organizational policies (e.g., MFA, conditional access).
- Tokens are validated server-side (issuer, audience, signature, expiration) on every request.
- Session handling is hardened (secure cookies where applicable; SameSite; CSRF protections if cookies are used).

Authorization (RBAC) and least privilege

- The backend enforces authorization for **every API** request; the UI does not imply authorization.
- App roles separate administrative actions (feature toggles, retention, safety review access) from standard user actions.
- Administrative actions are logged with actor identity, timestamp, and the requested change.

Supply chain security (client and dependencies)

No untrusted external JavaScript at runtime

- The application should avoid loading executable JavaScript from third-party CDNs at runtime.
- All scripts should be served from the application's controlled origin and versioned with the release artifact.
- A restrictive Content Security Policy (CSP) should disallow `unsafe-inline` and restrict `script-src` to self.

Dependency scanning and pinning

- Lockfiles pin transitive dependencies.
- CI runs SCA (dependency vulnerability scanning) and fails builds on high severity findings per policy.
- Builds are reproducible; artifacts are produced by controlled pipelines with protected branches and reviews.

API security

- Standardized auth middleware enforces authentication and role checks on all endpoints.
- Input validation and output encoding at API boundaries; structured error handling prevents information leakage.
- Rate limiting, request size limits, and workload quotas protect against abuse and resource exhaustion.
- Service-to-service calls use workload identity/managed identity where possible; secrets are minimized.

Transport security (encryption in transit)

- TLS is enforced for all user-facing and service-to-service endpoints.
- Modern TLS versions and strong cipher suites are required; HSTS is enabled for web endpoints.

Data protection (encryption at rest and beyond)

- Data at rest is encrypted for databases, storage, and search/index stores.
- Keys are managed in a KMS/HSM-backed service with rotation policies; customer-managed keys (CMK) can be used for higher assurance.
- Sensitive data is minimized; secrets and sensitive content are redacted from logs and telemetry.
- Backups are encrypted, access-controlled, and restore-tested periodically.

File upload and document ingestion security

- Server-side enforcement of file type and size constraints (allowlist/denylist).
- Malware scanning is **not performed explicitly by the application**. If the organization enables **Microsoft Defender for Cloud** and assigns it to the storage account (Defender for Storage), uploaded blobs can be scanned and flagged according to that service's configuration; any quarantine/response workflow should be implemented through organizational policy and automation.
- Storage is tenant/user scoped using strict ACLs; direct storage access uses short-lived signed URLs/tokens where used.
- Ingestion pipeline logs each step (extract, chunk, embed, index) with correlation IDs for investigations.

Operational security: deployment, monitoring, and incident readiness

- Infrastructure-as-Code (IaC) preferred; templates are reviewed and scanned for misconfiguration.
- Secrets are never committed; injected at runtime from a secrets manager.
- Network segmentation and private endpoints reduce exposure; public access is disabled where feasible.
- Monitoring and alerting detect auth anomalies, error spikes, ingestion failures, and suspicious access patterns.
- Incident response runbooks define triage, containment, eradication, and recovery steps; tabletop exercises are performed.